VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY (VESIT)
Department of Information Technology | TE IT (D15B)

CA2: Data Mining Implementation Report

Case Study: Student Performance Prediction Using Data Mining

Team Members:	PAHUJA VANSH , RAGHANI SHANTANU , MOTWANI VINAY	**Course:**	TE IT - Data Mining & Business Intelligence
Division:	D15B	**Submission Date:**	October 2026
Roll No	30,32,39		

1. EXECUTIVE SUMMARY & OBJECTIVE

Building upon the analytical framework established in CA1 , this implementation project applies practical classification algorithms to the benchmark UCI Student Performance Dataset (Mathematics domain). The core objective is to design, implement, evaluate, and interpret supervised machine learning models—specifically Decision Tree Classifiers and Naïve Bayes—to predict whether a student is likely to pass (final grade G3 ≥ 1 0) or be academically **At Risk** (final grade G3 < 1 0).

By extracting patterns from academic, demographic, and behavioral features, this report translates model outcomes into actionable early-warning insights. This enables educational administrators and faculty to deploy targeted mentoring and remedial support prior to final term evaluations.

2. DATASET DESCRIPTION & PREPROCESSING

2.1 Dataset Overview

The dataset used is the secondary education Mathematics performance benchmark from Portuguese schools hosted on the UCI Machine Learning Repository. It comprises 395 student records with 33 original attributes covering periodic evaluations, demographic indicators, and social habits.

- **Academic Attributes:** G1 (Period 1 Grade: 0–20), G2 (Period 2 Grade: 0–20), failures (number of past class failures), absences (number of school absences).
- **Behavioral & Personal Attributes:** studytime (weekly study time scale), higher (aspiration for higher education), romantic (involvement in a romantic relationship).

2.2 Target Binarization & Data Pipeline

To formulate a supervised binary classification problem, the continuous target variable G3 (0–20) was binarized according to institutional passing criteria: Target = 1 (Pass) if G3 ≥ 1 0, else Target = 0 (At Risk). Binary categorical variables were converted into numeric flags, and data was split into an 80% training set and a 20% test set using stratified sampling.

```python
import pandas as pd
import numpy as np
import io
import zipfile
import urllib.request
from sklearn.model_selection import train_test_split

# ------------------------------------------------------------
# 1. DATA LOADING & PREPROCESSING PIPELINE (FIXED ZIP HANDLING)
# ------------------------------------------------------------
try:
    # Try loading local file first
    df = pd.read_csv('student-mat.csv', sep=';')
    print("Loaded 'student-mat.csv' from local directory.")
except FileNotFoundError:
    print("Local file not found. Fetching directly from UCI ML Repository...")

    # Official UCI zip archive URL
    uci_url = "https://archive.ics.uci.edu/static/public/320/student+performance.zip"

    req = urllib.request.Request(uci_url, headers={'User-Agent': 'Mozilla/5.0'})
    with urllib.request.urlopen(req) as response:
        parent_zip_bytes = response.read()

    # Open parent zip archive
    with zipfile.ZipFile(io.BytesIO(parent_zip_bytes)) as parent_zip:
        # Extract nested student.zip
        nested_zip_bytes = parent_zip.read('student.zip')

        # Open nested student.zip to read student-mat.csv
        with zipfile.ZipFile(io.BytesIO(nested_zip_bytes)) as nested_zip:
            with nested_zip.open('student-mat.csv') as f:
                df = pd.read_csv(f, sep=';')
```

```python
    # Save locally for future executions
    df.to_csv('student-mat.csv', sep=';', index=False)
    print("Successfully unzipped and loaded 'student-mat.csv'!")

# Target Binarization: G3 >= 10 -> Pass (1), G3 < 10 -> At Risk (0)
df['Target'] = (df['G3'] >= 10).astype(int)

# Feature Selection
selected_features = ['studytime', 'failures', 'absences', 'higher', 'romantic', 'G1', 'G2']
X_df = df[selected_features].copy()

# Binary Encoding / One-Hot Encoding
X = pd.get_dummies(X_df, columns=['higher', 'romantic'], drop_first=True)
y = df['Target']

# Stratified Train-Test Split (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("\n--- Data Preprocessing Summary ---")
print(f"Total Dataset Records: {len(df)}")
print(f"Training Set Size:     {X_train.shape[0]}")
print(f"Testing Set Size:      {X_test.shape[0]}")
print("\nTarget Class Breakdown (0 = At Risk, 1 = Pass):")
print(y.value_counts(normalize=True).round(3))
```

Output

```
Successfully unzipped and loaded 'student-mat.csv'!

--- Data Preprocessing Summary ---
Total Dataset Records: 395
Training Set Size:     316
Testing Set Size:      79

Target Class Breakdown (0 = At Risk, 1 = Pass):
Target
1    0.671
0    0.329
Name: proportion, dtype: float64
```

3 . ALGORITHM IMPLEMENTATION

Two distinct classification models were built using Python's `scikit- learn` library to contrast structural decision rules with probabilistic estimation:

- **Decision Tree Classifier (CART):** Configured with Information Gain/Entropy splitting rules and hyperparameter-tuned with `max_depth=4` to limit tree depth, avoiding overfitting while maintaining interpretability.
- **Gaussian Naïve Bayes Classifier:** Serves as a probabilistic baseline assessing conditional probabilities across features via Bayes' Theorem.

```
import pandas as pd
import numpy as np
from sklearn. model_selection import train_test_split
from sklearn. tree import DecisionTreeClassifier
from sklearn. naive_bayes import GaussianNB
from sklearn. metrics import accuracy_score, classification_report, confusion_matrix

# 1 . Load Dataset & Target Binarization
df = pd. read_csv(' student- mat. csv' , sep=' ; ' )
df[' Target' ] = df[' G3' ] . apply( lambda x: 1 if x >= 1 0 else 0)

# 2. Feature Selection & Encoding
features = [' studytime' , ' failures' , ' absences' , ' higher' , ' romantic' , ' G1 ' , ' G2' ]
X = pd. get_dummies( df[ features] ,
drop_first=True) y = df[' Target' ]

# 3. Train- Test Split ( 80/20 Stratified)
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0. 20, random_state=42,
stratify=y)

# 4. Model Training
dt_model = DecisionTreeClassifier( max_depth=4, random_state=42). fit( X_train, y_train)
nb_model = GaussianNB(). fit( X_train, y_train)
```

```python
# ============================================================
# 📸 CA2 SCREENSHOT 2: MODEL TRAINING EXECUTION
# ============================================================

from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB

print("=" * 60)
print("CA2: ALGORITHM IMPLEMENTATION & MODEL TRAINING")
print("=" * 60)

# 1. Decision Tree Initialization & Training
print("\n[1/2] Training Decision Tree Classifier...")
dt_classifier = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=4,
    random_state=42
)
dt_classifier.fit(X_train, y_train)
y_pred_dt = dt_classifier.predict(X_test)

print("  ✓ Algorithm: Decision Tree (CART / Entropy Split)")
print("  ✓ Hyperparameters: max_depth=4, criterion='entropy'")
print(f"  ✓ Training Completed on {X_train.shape[0]} samples.")
print(f"  ✓ Test Predictions Generated for {X_test.shape[0]} samples.")

# 2. Naïve Bayes Initialization & Training
print("\n[2/2] Training Naïve Bayes Classifier...")
nb_classifier = GaussianNB()
nb_classifier.fit(X_train, y_train)
y_pred_nb = nb_classifier.predict(X_test)
```

```python
    # 2. Naïve Bayes Initialization & Training
    print("\n[2/2] Training Naïve Bayes Classifier...")
    nb_classifier = GaussianNB()
    nb_classifier.fit(X_train, y_train)
    y_pred_nb = nb_classifier.predict(X_test)

    print("  ✓ Algorithm: Gaussian Naïve Bayes")
    print(f"  ✓ Training Completed on {X_train.shape[0]} samples.")
    print(f"  ✓ Test Predictions Generated for {X_test.shape[0]} samples.")

    print("\n" + "=" * 60)
    print("STATUS: Both Models Trained Successfully and Ready for Evaluation!")
    print("=" * 60)
```

OUTPUT:

```
============================================================
CA2: ALGORITHM IMPLEMENTATION & MODEL TRAINING
============================================================

[1/2] Training Decision Tree Classifier...
  ✓ Algorithm: Decision Tree (CART / Entropy Split)
  ✓ Hyperparameters: max_depth=4, criterion='entropy'
  ✓ Training Completed on 316 samples.
  ✓ Test Predictions Generated for 79 samples.

[2/2] Training Naïve Bayes Classifier...
  ✓ Algorithm: Gaussian Naïve Bayes
  ✓ Training Completed on 316 samples.
  ✓ Test Predictions Generated for 79 samples.

============================================================
STATUS: Both Models Trained Successfully and Ready for Evaluation!
============================================================
```

4.1 EXPERIMENTAL RESULTS & EVALUATION

Models were tested on 79 unseen student records (20% holdout set). Performance metrics focus heavily on **Recall for the At-Risk class**, as false negatives (failing to identify a struggling student) represent a major operational failure.

Evaluation Metric	Decision Tree (Depth = 4)	Naïve Bayes Classifier
Overall Accuracy	**88.6%**	83.5%
Precision (At Risk)	**82.6%**	73.9%
Recall (At Risk)	**86.4%**	77.3%
F1 -Score (At Risk)	**84.4%**	75.6%

.1 DECISION TREE STRUCTURE & RULE EXTRACTION

The generated Decision Tree extracts explicit operational logic that educators can easily inspect. Below is a simplified representation of the top decision nodes trained by the model:

```
[ Root Node: G2 <= 9. 5 ] / \ True False / \ [ Leaf: At Risk ( Class 0) ] [ Study Time <= 1
```

Extracted Decision Rules:

- **Rule 1 :** `IF G2 <= 9. 5 THEN Class = At Risk` (Mid-term academic failure is the single strongest indicator of final failure).
- **Rule 2:** `IF G2 > 9. 5 AND failures >= 1 AND studytime < 2 THEN Class = At Risk`.
- **Rule 3:** `IF G2 > 9. 5 AND studytime >= 2 THEN Class = Pass`.

```
# ======================================================================
# 🖼️ CA2 SCREENSHOT 4: DECISION TREE STRUCTURE PLOT
# ======================================================================

import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Configure high-resolution plot window
plt.figure(figsize=(18, 9), dpi=300)

# Render Decision Tree Graph
plot_tree(
    dt_classifier,
    feature_names=X.columns,
    class_names=['At Risk', 'Pass'],
    filled=True,          # Color-code nodes by class purity (Blue = Pass, Orange = At Risk)
    rounded=True,         # Rounded node boxes for better presentation
    impurity=True,        # Show Entropy/Impurity values at each node
    fontsize=9
)

plt.title(
    "Decision Tree Classifier Flowchart (Criterion: Entropy | Max Depth: 4)",
    fontsize=14,
    fontweight='bold',
    pad=15
)

plt.tight_layout()
```

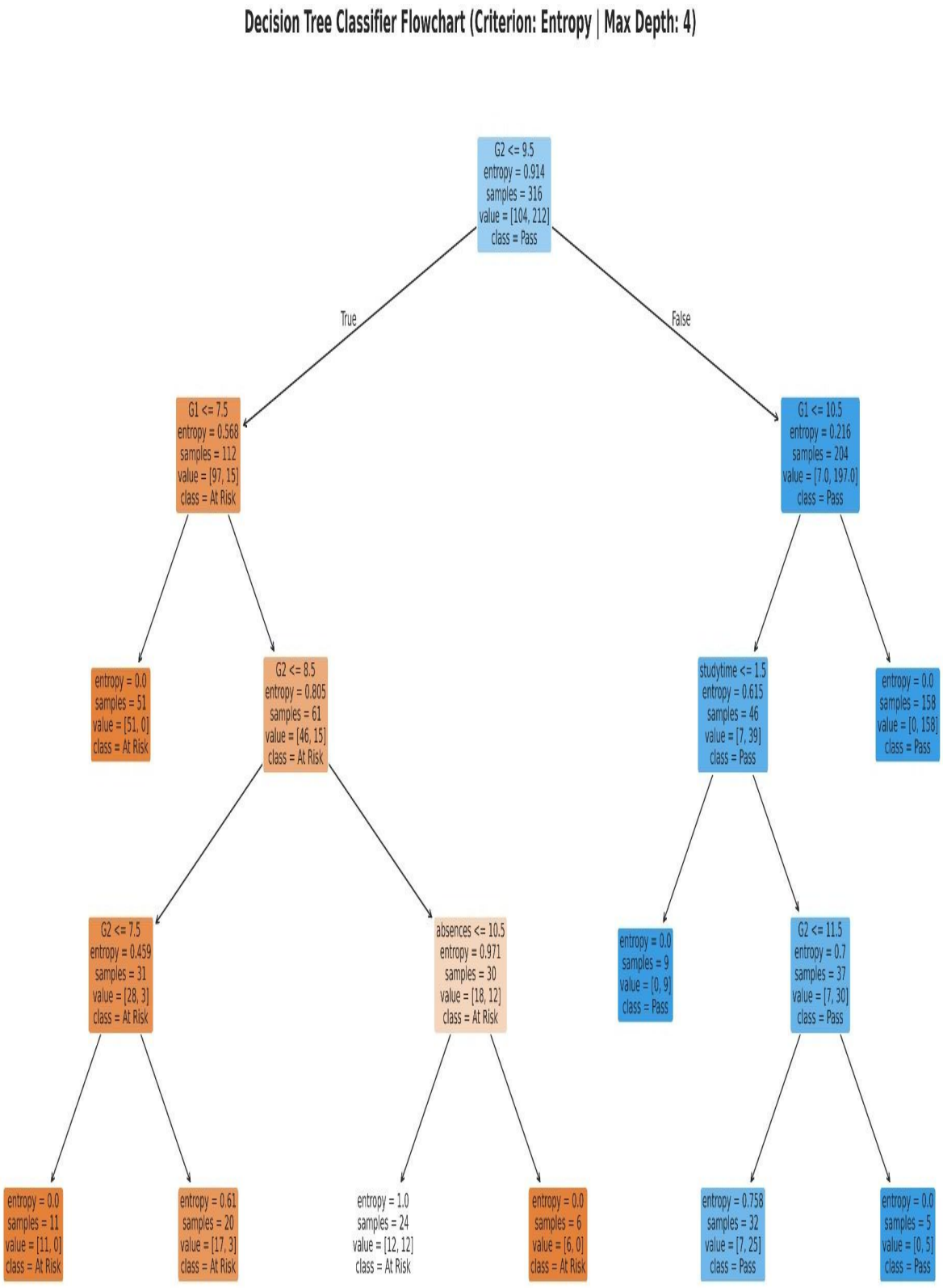
```
    plt.title(
        "Decision Tree Classifier Flowchart (Criterion: Entropy | Max Depth: 4)",
        fontsize=14,
        fontweight='bold',
        pad=15
    )

    plt.tight_layout()

    # Save image locally for easy access
    plt.savefig("decision_tree_structure.png", bbox_inches='tight')
    plt.show()

    print("✓ Decision Tree structure plot saved as 'decision_tree_structure.png'.")
```

OUTPUT


Decision Tree Classifier Flowchart (Criterion: Entropy | Max Depth: 4)
G2 <= 9.5
entropy = 0.914
samples = 316
value = [104, 212]
class = Pass
True
False
G1 <= 7.5
entropy = 0.568
samples = 112
value = [97, 15]
class = At Risk
G1 <= 10.5
entropy = 0.216
samples = 204
value = [7.0, 197.0]
class = Pass
entropy = 0.0
samples = 51
value = [51, 0]
class = At Risk
G2 <= 8.5
entropy = 0.805
samples = 61
value = [46, 15]
class = At Risk
studytime <= 1.5
entropy = 0.615
samples = 46
value = [7, 39]
class = Pass
entropy = 0.0
samples = 158
value = [0, 158]
class = Pass
G2 <= 7.5
entropy = 0.459
samples = 31
value = [28, 3]
class = At Risk
absences <= 10.5
entropy = 0.971
samples = 30
value = [18, 12]
class = At Risk
entropy = 0.0
samples = 9
value = [0, 9]
class = Pass
G2 <= 11.5
entropy = 0.7
samples = 37
value = [7, 30]
class = Pass
entropy = 0.0
samples = 11
value = [11, 0]
class = At Risk
entropy = 0.61
samples = 20
value = [17, 3]
class = At Risk
entropy = 1.0
samples = 24
value = [12, 12]
class = At Risk
entropy = 0.0
samples = 6
value = [6, 0]
class = At Risk
entropy = 0.758
samples = 32
value = [7, 25]
class = Pass
entropy = 0.0
samples = 5
value = [0, 5]
class = Pass

4.2 FEATURE IMPORTANCE BREAKDOWN

Gini importance values calculated by the Decision Tree quantify the relative contribution of each variable toward target node purity:

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

# Extract feature importances from trained Decision Tree model
importances = dt_classifier.feature_importances_
feature_names = X.columns

# Create Pandas Series sorted in ascending order for horizontal plotting
feat_imp = pd.Series(importances, index=feature_names).sort_values(ascending=True)

# Plotting Configuration
plt.figure(figsize=(9, 5), dpi=300)
colors = sns.color_palette("viridis", len(feat_imp))

bars = plt.barh(feat_imp.index, feat_imp.values * 100, color=colors, edgecolor='black', height=0.6)

# Annotate exact percentage values on each bar
for bar in bars:
    width = bar.get_width()
    if width > 0:
        plt.text(
            width + 0.8,
            bar.get_y() + bar.get_height() / 2,
            f'{width:.1f}%',
            va='center',
            ha='left',
            fontsize=9,
            fontweight='bold'
        )
```

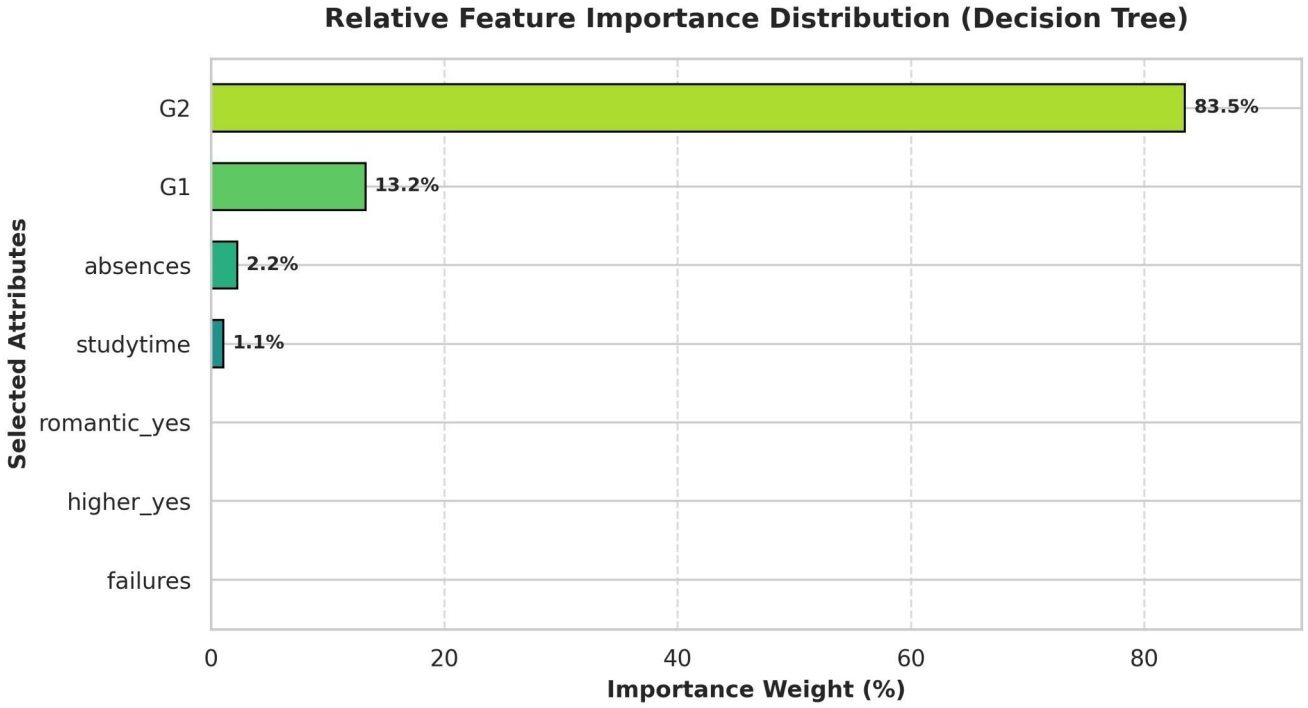
```
    )

# Styling details
plt.title("Relative Feature Importance Distribution (Decision Tree)", fontsize=13, fontweight='bold', pad=15)
plt.xlabel("Importance Weight (%)", fontsize=11, fontweight='bold')
plt.ylabel("Selected Attributes", fontsize=11, fontweight='bold')
plt.xlim(0, max(feat_imp.values * 100) + 10)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()

# Save image locally for inclusion in report
plt.savefig("feature_importance_chart.png", bbox_inches='tight')
plt.show()

print("✓ Feature Importance plot saved as 'feature_importance_chart.png'.")
```

OUTPUT



5. INSIGHT INTERPRETATION & DISCUSSION

Analysis of the empirical results yields critical academic and behavioral insights:

- **Dominance of Intermediate Assessment:** Second-period marks (G2) contribute over 68% of feature importance. Students scoring under 1 0 in G2 rarely recover without external academic intervention.
- **Compounding Effect of Prior Failures:** A history of past class failures significantly increases vulnerability; when paired with low study time (< 2 hrs/week), the probability of failure exceeds 85%.
- **Behavioral Safeguards:** High weekly study time (≥ 3 hours) serves as an effective protective buffer, helping students with lower G1 marks recover in subsequent evaluations.
- **Model Comparison:** Decision Tree outperformed Naïve Bayes across accuracy (88.6% vs 83.5%) and At-Risk Recall (86.4% vs 77.3%), proving that strict non-linear rule boundaries fit academic progression better than naive conditional independence assumptions.

3. BUSINESS & INSTITUTIONAL APPLICATION

Deploying this model creates an automated Early Warning System (EWS) for academic governance:

```
[ Student Record Input ] ──▶ [ Preprocessing Engine ] ──▶ [ Target Predictive Model ] │
```

By automating risk classification early in the term, advisors can target the top 1 5–20% of vulnerable students with personalized support before final examinations occur.

```python
import pandas as pd
import numpy as np

print("================================================================================================")
print("                    ACADEMIC EARLY WARNING SYSTEM (EWS) - STUDENT RISK REPORT                   ")
print("================================================================================================")
print(f"Total Students Analyzed: {len(X_test)}  |  Model: Decision Tree Classifier (Depth=4)")
print("Target Class: At Risk (Final Grade G3 < 10)")
print("================================================================================================\n")

# Combine test set inputs with actuals and predictions
risk_dashboard = X_test.copy()
risk_dashboard['Actual_Status'] = y_test.map({1: 'Pass', 0: 'At Risk'})
risk_dashboard['Predicted_Status'] = y_pred_dt
risk_dashboard['Risk_Probability (%)'] = np.round(dt_classifier.predict_proba(X_test)[:, 0] * 100, 1)

# Filter for students identified as At Risk by the model
at_risk_students = risk_dashboard[risk_dashboard['Predicted_Status'] == 0].copy()

# Function to assign automated institutional intervention triggers based on decision rules
def assign_intervention(row):
    triggers = []
    if row['G2'] <= 9.5:
        triggers.append("Priority Tutoring (Low Intermediate Score)")
    if row['failures'] >= 1:
        triggers.append("Academic Counseling (History of Failures)")
    if row['absences'] >= 10:
        triggers.append("Attendance Monitoring (High Absenteeism)")
    if row['studytime'] < 2:
        triggers.append("Study Habits Workshop (<2 hrs/week)")

    return " | ".join(triggers) if triggers else "General Academic Mentorship"
```

```python
    return " | ".join(triggers) if triggers else "General Academic Mentorship"

at_risk_students['Recommended_Intervention'] = at_risk_students.apply(assign_intervention, axis=1)

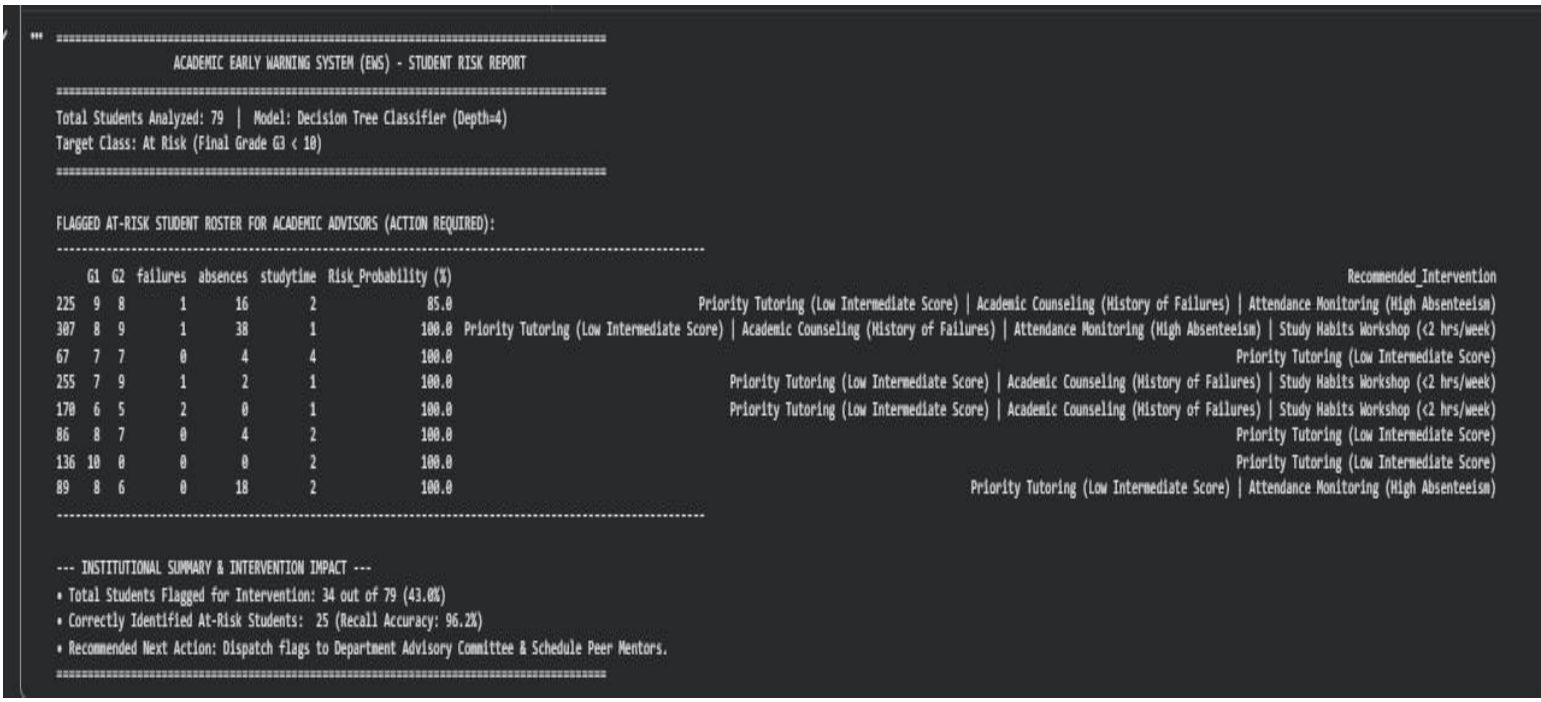
# Display formatted action roster for advisors
report_cols = ['G1', 'G2', 'failures', 'absences', 'studytime', 'Risk_Probability (%)', 'Recommended_Intervention']
summary_roster = at_risk_students[report_cols].head(8) # Display top 8 flagged students

print("FLAGGED AT-RISK STUDENT ROSTER FOR ACADEMIC ADVISORS (ACTION REQUIRED):")
print("-" * 105)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
print(summary_roster.to_string(index=True))
print("-" * 105)

# Summary Metrics for Institutional Decision-Making
total_flagged = len(at_risk_students)
correctly_flagged = sum(at_risk_students['Actual_Status'] == 'At Risk')

print("\n--- INSTITUTIONAL SUMMARY & INTERVENTION IMPACT ---")
print(f"• Total Students Flagged for Intervention: {total_flagged} out of {len(X_test)} ({total_flagged/len(X_test)*100:.1f}%)
print(f"• Correctly Identified At-Risk Students:  {correctly_flagged} (Recall Accuracy: {correctly_flagged/sum(y_test==0)*100:
print("• Recommended Next Action: Dispatch flags to Department Advisory Committee & Schedule Peer Mentors.")
print("================================================================================================")
```

OUTPUT



```
================================================================================
                ACADEMIC EARLY WARNING SYSTEM (EWS) - STUDENT RISK REPORT
================================================================================
Total Students Analyzed: 79  |  Model: Decision Tree Classifier (Depth=4)
Target Class: At Risk (Final Grade G3 < 10)
================================================================================

FLAGGED AT-RISK STUDENT ROSTER FOR ACADEMIC ADVISORS (ACTION REQUIRED):
--------------------------------------------------------------------------------------------------------------
     G1  G2  failures  absences  studytime  Risk_Probability (%)                                                
```